Programação concorrente

1º semestre de 2026

Speedup

Tema #06

Professor Marcelo Eustáquio

Speedup representa o ganho de desempenho obtido ao executar uma tarefa com múltiplos processadores em comparação à execução sequencial.

Marcelo Eustáquio

Speedup

Speedup, ou **aceleração**, é uma métrica que mede o ganho de desempenho de um sistema ao executar uma tarefa em paralelo em comparação à sua execução sequencial e é definido pela razão entre o tempo de execução do melhor algoritmo sequencial e o tempo de execução do algoritmo paralelo.

$$S(N) = \frac{T(1)}{T(N)}$$

Onde:

- $S(n)$ é o speedup com n processadores.
- $T(1)$ é o tempo necessário para executar a tarefa em um único processador (de forma sequencial).
- $T(n)$ é o tempo necessário para executar a tarefa em n processadores (de forma paralela).

S(n) é utilizado para avaliar a eficácia e a eficiência de algoritmos que utilizam múltiplos processadores.

Speedup

Um algoritmo leva 40 segundos em execução sequencial e 12 segundos com 4 processadores. Determine o speedup desse algoritmo.

Como são 4 núcleos, o **speedup** deveria ser 4 (speedup ideal). Por que isso não acontece?

Speedup

Normalmente, o *speedup ideal* é diferente do *speedup real* ($S_{ideal} \neq S_{real}$). Isso ocorre por causa de:

1	Overhead de criação de threads.
2	Sincronização e mecanismos de bloqueio.
3	Comunicação entre processos.
3	Espera por recursos.
5	Parte sequencial do algoritmo.

Speedup

Speedup, ou **aceleração**, é uma métrica que mede o ganho de desempenho de um sistema ao executar uma tarefa em paralelo em comparação à sua execução sequencial e é definido pela razão entre o tempo de execução do melhor algoritmo sequencial e o tempo de execução do algoritmo paralelo.

$$S(N) = \frac{T(1)}{T(N)}$$

Interpretação:

- $S(n) > 1$ indica que a versão paralela é mais rápida que a sequencial.
- $S(n) = 1$ indica que a versão paralela tem o mesmo desempenho que a versão sequencial.
- $S(n) < 1$ indica que a versão paralela é mais lenta que a versão sequencial.

Speedup

Calcule o speedup do algoritmo a seguir:

```
import time
import threading

resultado = []

def quadrado_lista(parte):
    global resultado
    for x in parte:
        resultado.append(x * x)

def versao_serial(dados):
    return [x * x for x in dados]
```

```python
def versao_threading(dados, num_threads = 4):

    global resultado
    resultado = []

    tamanho = len(dados) // num_threads
    threads = []

    for i in range(num_threads):

        inicio = i * tamanho
        fim = len(dados) if i == num_threads - 1 else (i + 1) * tamanho

        t = threading.Thread(target=quadrado_lista, args=(dados[inicio:fim],) )
        threads.append(t)
        t.start()


    for t in threads: t.join()

    return resultado
```

```python
dados = list(range(1000))  # carga pequena

inicio = time.perf_counter()
serial = versao_serial(dados)
fim = time.perf_counter()
tempo_serial = fim - inicio

inicio = time.perf_counter()
paralelo = versao_threading(dados)
fim = time.perf_counter()
tempo_thread = fim - inicio

print(f"Tempo serial:   {tempo_serial:.6f} s")
print(f"Tempo threading:{tempo_thread:.6f} s")
```

Speedup

Por definição, o speedup **S(N)** é calculado por:

$$S(N) = \frac{T(1)}{T(N)}$$

Em função da proporção entre S(N) e o número de processadores, o speedup pode ser classificado em **linear**, **sublinear** e **superlinear**.

1	O speedup é **linear** se o ganho cresce **na mesma proporção** do n° de processadores (**S = p**).
2	O speedup é **sublinear** se o ganho é **menor** que a quantidade de processadores (**S < p**).
3	O speedup é **superlinear** se o ganho é **maior** que a quantidade de processadores (**S > p**).

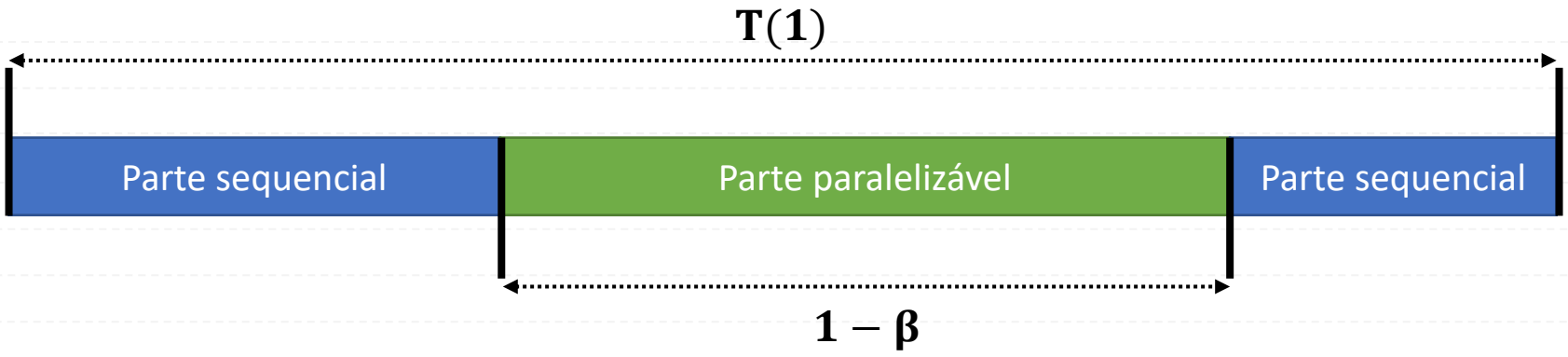
Speedup

Para cada situação apresentada a seguir, classifique o speedup como linear, sublinear e superlinear.

Processadores	1	2	4	8
Tempo (s)	100	55	28	20

Lei de Amdahl

A **Lei de Amdahl**, formulada por Gene Amdahl em 1967, é um princípio descreve a potencial aceleração de um sistema quando uma parte do processo é paralelizada. A lei é usada para estimar o ganho máximo de desempenho que pode ser alcançado ao aplicar maior paralelismo a uma tarefa.

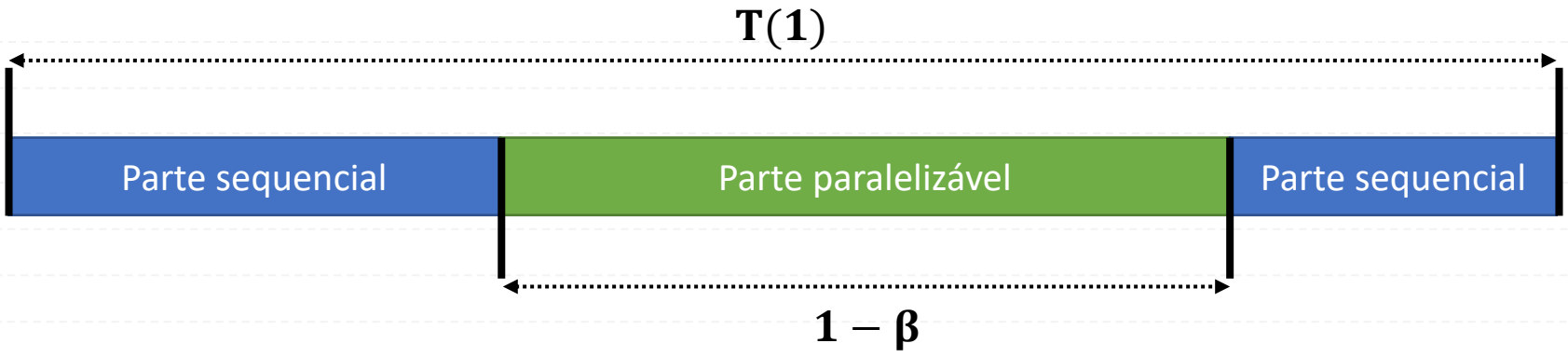


Em que $\boldsymbol{\beta}$ é a fração do código que é puramente sequencial e $1 - \boldsymbol{\beta}$ é a fração do código paralelizável. Caso esse código seja executado usando **2 processadores**, o tempo de execução é dado por:

$$T(2) = \beta \cdot T(1) + (1 - \beta) \cdot \frac{T(1)}{2}$$

Lei de Amdahl

A **Lei de Amdahl**, formulada por Gene Amdahl em 1967, é um princípio descreve a potencial aceleração de um sistema quando uma parte do processo é paralelizada. A lei é usada para estimar o ganho máximo de desempenho que pode ser alcançado ao aplicar maior paralelismo a uma tarefa.

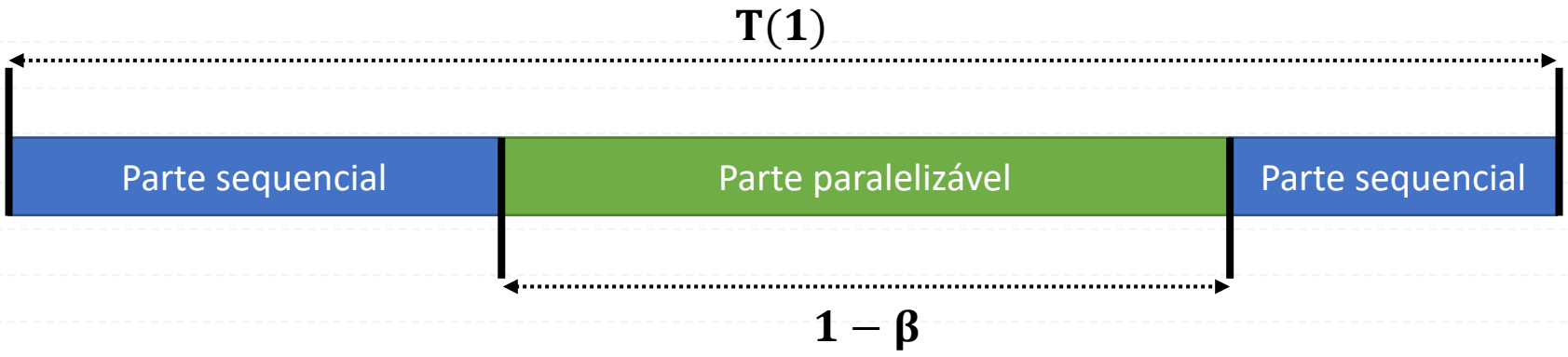


Em que $\boldsymbol{\beta}$ é a fração do código que é puramente sequencial e $1 - \boldsymbol{\beta}$ é a fração do código paralelizável. Passando a usar **3 processadores**, o tempo de execução pode ser expresso por:

$$T(3) = \beta \cdot T(1) + (1 - \beta) \cdot \frac{T(1)}{3}$$

Lei de Amdahl

A **Lei de Amdahl**, formulada por Gene Amdahl em 1967, é um princípio descreve a potencial aceleração de um sistema quando uma parte do processo é paralelizada. A lei é usada para estimar o ganho máximo de desempenho que pode ser alcançado ao aplicar maior paralelismo a uma tarefa.



Em que $\boldsymbol{\beta}$ é a fração do código que é puramente sequencial e $1 - \boldsymbol{\beta}$ é a fração do código paralelizável. Por fim, generalizando para **N processadores**, obtemos:

$$T(N) = \beta \cdot T(1) + (1 - \beta) \cdot \frac{T(1)}{N}$$

Lei de Amdahl

Seja $\boldsymbol{\beta}$ a fração do código que é sequencial e $1 - \boldsymbol{\beta}$ é a fração do código paralelizável. A lei de Amdahl estabelece que o speedup máximo que uma aplicação paralela com N processadores pode obter é:

$$S(N) = \frac{1}{\beta + \frac{(1-\beta)}{N}}$$

Demonstração:

Basta usar a definição de speedup e processo para determinar $T(N)$:

$$S(N) = \frac{T(1)}{T(N)} = \frac{T(1)}{\beta \cdot T(1) + (1-\beta) \cdot \frac{T(1)}{N}} = \frac{T(1)}{T(1) \cdot \left[\beta + \frac{(1-\beta)}{N}\right]}$$

Lei de Amdahl

Um programa, em execução usado 4 processadores (n = 4), tem 80% (P = 0,80) de seu código paralelizado. Determine o ganho máximo que se obtém com a paralelização.

Lei de Amdahl

(FGV – 2024 – INPE) Considere uma aplicação que será executada em um sistema com múltiplos núcleos de computação. Segundo a Lei de Amdahl, o limite máximo de aceleração, em relação a um sistema com um único núcleo, considerando que 20% dessa aplicação será executada em paralelo, é de (em vezes)

A) 1,25.
B) 2,25.
C) 3,25.
D) 3,50.
E) 5,00.

Lei de Amdahl

Este exercício tem por finalidade implementar um simulador (em Python) para calcular e visualizar o speedup com base nos valores de f e de p.

```python
import matplotlib.pyplot as plt

def amdahl_speedup(f, p_values):
    results = [(p, 1 / ((1 - f) + f / p)) for p in p_values]
    return results

f = 0.6
p_values = [1, 2, 4, 8, 16, 32]

speedups = amdahl_speedup(f, p_values)

# Exibindo os resultados
print("p\tSpeedup")
for p, speedup in speedups:
    print(f"{p}\t{speedup:.2f}")

# Gráfico
plt.plot(p_values, [s[1] for s in speedups], marker='o')
plt.xlabel("Número de Processadores (p)")
plt.ylabel("Speedup (S(p))")
plt.title(f"Speedup vs Processadores para f = {f}")
plt.grid()
plt.show()
```

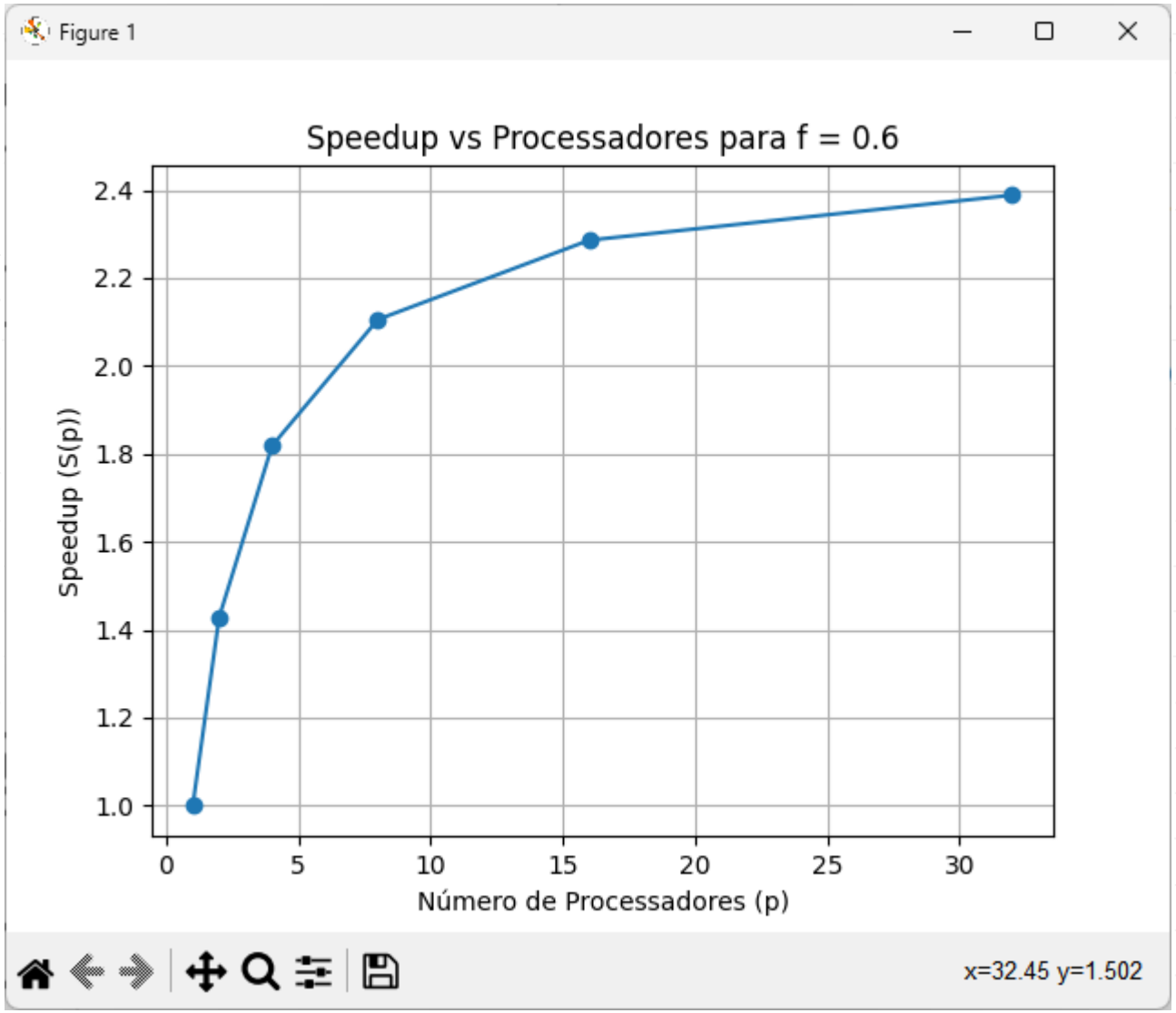


Figure 1
Speedup vs Processadores para f = 0.6
Speedup (S(p))
2.4
2.2
2.0
1.8
1.6
1.4
1.2
1.0
0
5
10
15
20
25
30
Número de Processadores (p)
x=32.45 y=1.502

Speedup

A variância é um conceito, no âmbito da estatística e da probabilidade, usado para medir a dispersão ou variabilidade de um conjunto de dados em relação à sua média. De forma simplificada, a variância quantifica o quão espalhados os elementos de um conjunto estão ao redor do valor médio.

Definição:

Para um conjunto de dados $X = \{x_1, x_2, x_3, \dots, x_n\}$, a variância (σ^2) é definida como a média aritmética dos quadrados dos desvios (diferenças) entre cada elemento e a média do conjunto. Matematicamente, escrevemos:

$$\sigma^2 = \frac{(x_1 - \mu)^2 + (x_2 - \mu)^2 + \cdots + (x_n - \mu)^2}{n} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2$$

Speedup

A variância é um conceito, no âmbito da estatística e da probabilidade, usado para medir a dispersão ou variabilidade de um conjunto de dados em relação à sua média. De forma simplificada, a variância quantifica o quão espalhados os elementos de um conjunto estão ao redor do valor médio.

Exemplo:

Calculando a variância dos elementos do conjunto {2, 4, 6, 8, 10}.

1	A média dos elementos é $\mu = \frac{2+4+6+8+10}{5} = \frac{30}{5} = 6$ (coincidentemente, igual ao termo central).
2	Os desvio e os desvios quadrados em relação a μ são $\{-4, -2, 0, 2, 4\}$ e $\{16, 4, 0, 4, 16\}$.
3	Logo, a variância é $\sigma^2 = \frac{16+4+0+4+16}{5} = \frac{40}{5} = 8$

Speedup

Em Python, uma forma de implementar uma função para calcular a variância dos elementos de um conjunto é mostrada a seguir:

```
def CalcularVariancia(Dados):

    N = len(dados)

    Soma = 0
    for x in Dados: Soma = Soma + x
    Media = Soma / N

    SomaDQ = 0
    for x in Dados: SomaDQ = SomaDQ + (x - Media) ** 2
    Variancia = Soma / N  # Variância (populacional)

    return Variancia
```

Speedup

Em Python, uma forma de implementar uma função para calcular a variância dos elementos de um conjunto é mostrada a seguir:

```python
# Exemplo de uso

dados = [2, 4, 6, 8, 10]

# Calcular a variância populacional

V = CalcularVariancia(dados)
print(f"Variância Populacional: {V}")
```

```
C:\Users\Administrador>C:/Python311/python.exe c:/Users/Administrador/Desktop/Untitled-1.py
Variância Populacional: 6.0
```

Atenção!!!

Uma forma de paralelizar o algoritmo proposto está em criar uma função para “somar quadrados das diferenças” para manipular parte do vetor original.

```
def SomaQuadrados(Dados, Media):

    N = len(dados)
    Soma = 0
    for x in Dados: Soma = Soma + (x - Media) ** 2
    return Soma
```

... e esta será a única paralelização que faremos!

Speedup

Enfim, a versão paralelizada...

```
def VarianciaParalelizada(Dados):

    N = len(Dados)
    Meio = N // 2
    Media = sum(Dados) / N
    Esquerda = Dados[:Meio]
    Direita = Dados[Meio:]
```

Como estratégia, vamos criar funções para somar os “desvios quadrados” dos subconjuntos Esquerda e Direita, a serem desenvolvidos em threads diferentes. A vinculação desses resultados com o restante da função se dará em função do modificador de acesso ***nonlocal***, que é utilizado para indicar que uma variável pertence a um escopo externo, mas não global.

```python
SQEsquerda = 0
SQDireita = 0
def ProcessarEsquerda():
    nonlocal SQEsquerda
    SQEsquerda = SomaQuadrados(Esquerda, Media)
def ProcessarDireita():
    nonlocal SQDireita
    SQDireita = SomaQuadrados(Direita, Media)

TE = threading.Thread(target = ProcessarEsquerda)
TD = threading.Thread(target = ProcessarDireita)
TE.start()
TD.start()
TE.join()
TD.join()

# Combinando resultados

SQTotal = SQDireita + SQEsquerda
Variancia = SQTotal / N
return Variancia
```

Por fim, o bloco principal (“***main***”), que não se encontra dentro de nenhuma função.

```
# Exemplo de uso

dados = [2, 4, 6, 8, 10]

# Calcular a variância populacional

V = VarianciaParalelizada(dados)
print(f"Variância Populacional: {V}")
```

E então, o que é speedup, que é o conceito principal da aula de hoje?

Speedup

Speedup, ou **aceleração**, é uma métrica que mede o ganho de desempenho de um sistema ao executar uma tarefa em paralelo em comparação à sua execução sequencial e é definido pela razão entre o tempo de execução do melhor algoritmo sequencial e o tempo de execução do algoritmo paralelo.

```
t0 = time.time()
V = CalcularVariancia(dados)
tf = time.time()
T1 = tf - t0

t0 = time.time()
V = VarianciaParalelizada(dados)
tf = time.time()
Tn = tf - t0

SpeedUP = T1/Tn
```

Speedup

A função ***tarefa_io*** simula uma operação I/O-bound usando time.sleep(2), que simula uma operação que leva tempo, como acessar um banco de dados, fazer uma requisição HTTP ou ler/gravar arquivos.

```
import time

def tarefa_io(id_tarefa):
    print(f"Tarefa {id_tarefa} iniciada.")
    time.sleep(2)  # Simula uma operação I/O que leva 2 segundos
    print(f"Tarefa {id_tarefa} concluída.")
```

Speedup

Considere a seguinte versão (sequencial) que simula tarefas sequenciais...

```
import time

def main_sequencial():
    start_time = time.time()
    for i in range(4):
        tarefa_io(i)
    end_time = time.time()
    print(f"Tempo total (sequencial): {end_time - start_time:.2f} segundos")

# Executar a versão sequencial
main_sequencial()
```

Speedup

Agora, a versão paralela...

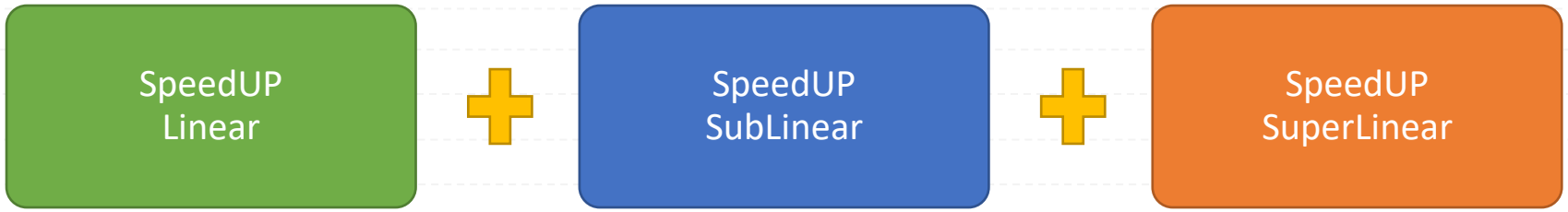
```
def main_paralela():
    start_time = time.time()
    threads = []

    for i in range(4):
        thread = threading.Thread(target=tarefa_io, args=(i,))
        threads.append(thread)
        thread.start()

    for thread in threads:
        thread.join()

    end_time = time.time()
    print(f"Tempo total (paralela): {end_time - start_time:.2f} segundos")

# Executar a versão paralela
main_paralela()
```

Speedup

Qual o SpeedUP dessa situação?

Tipos de speedup

Speedup pode ser classificado em linear, sublinear e superlinear.



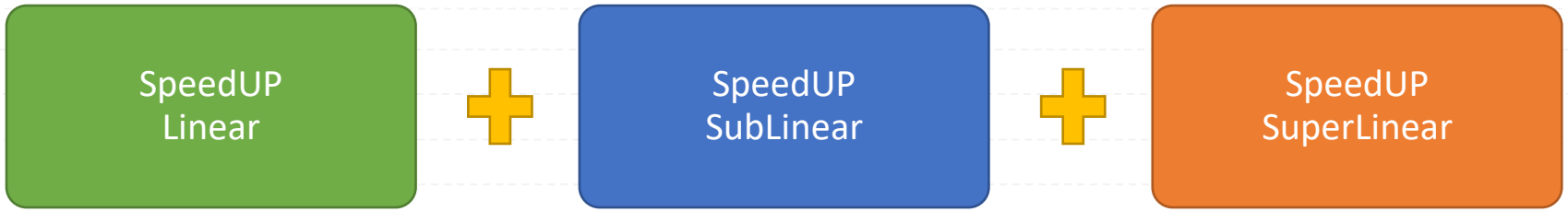
Speedup Linear (**Ideal**) é o caso ideal em que o aumento do número de processadores resulta em uma redução proporcional no tempo de execução. Se o speedup é linear, duplicar o número de processadores reduz o tempo pela metade, triplicar reduz a um terço, e assim por diante.

Exemplo:

Um speedup linear ocorre teoricamente se o problema puder ser perfeitamente dividido em partes independentes, com zero sobrecarga de comunicação e sincronização entre processadores.

Tipos de speedup

Speedup pode ser classificado em linear, sublinear e superlinear.



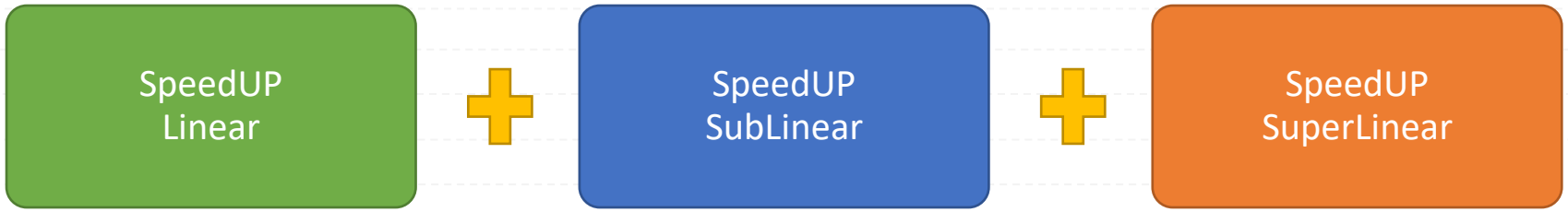
Speedup Superlinear ocorre quando o speedup é maior que o número de processadores. Embora incomum, pode acontecer em casos específicos, como quando o uso de memória cache por vários processadores resulta em uma otimização inesperada.

Exemplo:

Em alguns algoritmos de busca, o paralelismo permite que certos processadores encontrem uma solução mais cedo, interrompendo a execução dos demais.

Tipos de speedup

Speedup pode ser classificado em linear, sublinear e superlinear.



Speedup Sublinear ocorre quando o aumento no número de processadores não leva a um aumento proporcional na performance devido a fatores como sobrecarga de comunicação, sincronização e partes não paralelizáveis do algoritmo.

Exemplo:

É a situação mais comum e ocorre em algoritmos que possuem seções sequenciais inevitáveis e, consequentemente, a paralelização completa é impossível, reduzindo a eficiência.

Lei de Amdahl

1. Suponha que 70% de um programa pode ser paralelizado. Calcule o speedup S(p) para $p \in \{2,4,8\}$.
2. Qual o limite teórico de aceleração $S(\infty)$ para este programa?
3. O que acontece com o speedup quando f = 0,50 e $\mathrm{p} \to \infty$?

Lei de Amdahl

1. Suponha que 70% de um programa pode ser paralelizado. Calcule o speedup S(p) para $p \in \{2,4,8\}$.
2. Qual o limite teórico de aceleração $S(\infty)$ para este programa?
3. O que acontece com o speedup quando f = 0,50 e p → ∞?

O speedup $S(p)$ é definido por $S(p) = \frac{1}{(1-f)+\frac{f}{p}}$. Substituindo $f = 0{,}70$, $p = 2$, $p = 4$ e $p = 8$, tem-se:

$$S(2) = \frac{1}{(1-0{,}7)+\frac{0{,}7}{2}} = \frac{1}{0{,}30+0{,}35} = \frac{1}{0{,}65} \approx 1{,}54.$$

$$S(4) = \frac{1}{(1-0{,}7)+\frac{0{,}7}{4}} = \frac{1}{0{,}30+0{,}175} = \frac{1}{0{,}475} \approx 2{,}11.$$

$$S(8) = \frac{1}{(1-0{,}7)+\frac{0{,}7}{8}} = \frac{1}{0{,}30+0{,}0875} = \frac{1}{0{,}3875} \approx 2{,}58.$$

Lei de Amdahl

1. Suponha que 70% de um programa pode ser paralelizado. Calcule o speedup S(p) para $p \in \{2,4,8\}$.
2. Qual o limite teórico de aceleração $S(\infty)$ para este programa?
3. O que acontece com o speedup quando f = 0,50 e p → ∞?

O speedup $S(p)$ é definido por $S(p) = \frac{1}{(1-f)+\frac{f}{p}}$.

Para p suficientemente grande, ou seja, para $p \to \infty$, tem-se que o quociente $\mathrm{p/f} \to 0$ e, consequentemente, que $S(p) = 1/(1-f)$. Logo:

$$S(\infty) = \frac{1}{(1-0{,}7)} = \frac{1}{0{,}30} \approx 3{,}33.$$

Lei de Amdahl

1. Suponha que 70% de um programa pode ser paralelizado. Calcule o speedup S(p) para $p \in \{2,4,8\}$.
2. Qual o limite teórico de aceleração $S(\infty)$ para este programa?
3. O que acontece com o speedup quando f = 0,50 e $\mathrm{p} \to \infty$?

O speedup $S(p)$ é definido por $S(p) = \frac{1}{(1-f)+\frac{f}{p}}$.

Para p suficientemente grande, ou seja, para $p \to \infty$, tem-se que o quociente $\mathrm{p/f} \to 0$ e, consequentemente, que $S(p) = 1/(1-f)$. Logo:

$$S(\infty) = \frac{1}{(1-0{,}5)} = \frac{1}{0{,}50} \approx 2{,}00.$$

Programação concorrente

1º semestre de 2025

Speedup

Exercícios

Professor Marcelo Eustáquio

```python
import time

def soma_sequencial(dados):
    total = 0
    for numero in dados:
        total += numero
    return total

# Exemplo de uso
dados = list(range(1, 1000001))  # Lista de 1 milhão de números
start_time = time.time()
resultado = soma_sequencial(dados)
end_time = time.time()

print(f"Resultado: {resultado}")
print(f"Tempo total (sequencial): {end_time - start_time:.2f} segundos")
```

```python
import time
from multiprocessing import Pool

def soma_parcial(dados):
    return sum(dados)

def soma_paralela(dados, num_processos=4):
    pool = Pool(processes=num_processos)
    tamanho_parte = len(dados) // num_processos
    partes = [dados[i * tamanho_parte:(i + 1) * tamanho_partes] for i in range(num_processos)]
    resultados = pool.map(soma_parcial, partes)
    pool.close()
    pool.join()
    return sum(resultados)
# Exemplo de uso
dados = list(range(1, 1000001))  # Lista de 1 milhão de números
start_time = time.time()
resultado = soma_paralela(dados)
end_time = time.time()

print(f"Resultado: {resultado}")
print(f"Tempo total (paralela): {end_time - start_time:.2f} segundos")
```

Q01 Determine o speedup dos dois códigos anteriores.

```python
import time

def processar_strings_sequencial(strings):
    resultado = []
    for s in strings:
        resultado.append(s.upper())  # Converte string para maiúsculas
    return resultado

# Exemplo de uso
strings = ["abc"] * 1000000  # 1 milhão de strings
start_time = time.time()
resultado = processar_strings_sequencial(strings)
end_time = time.time()

print(f"Primeiros 5 resultados: {resultado[:5]}")
print(f"Tempo total (sequencial): {end_time - start_time:.2f} segundos")
```

```python
import time
import threading

def processar_strings_paralela(strings, num_threads=4):
    resultado = [None] * len(strings)
    lock = threading.Lock()
    def processar_parte(inicio, fim):
        for i in range(inicio, fim):
            resultado[i] = strings[i].upper()
    threads = []
    tamanho_parte = len(strings) // num_threads
    for i in range(num_threads):
        inicio = i * tamanho_parte
        fim = (i + 1) * tamanho_parte if i < num_threads - 1 else len(strings)
        thread = threading.Thread(target=processar_parte, args=(inicio, fim))
        threads.append(thread)
        thread.start()
    for thread in threads:
        thread.join()

    return resultado
```

```python
# Exemplo de uso
strings = ["abc"] * 1000000  # 1 milhão de strings
start_time = time.time()
resultado = processar_strings_paralela(strings)
end_time = time.time()

print(f"Primeiros 5 resultados: {resultado[:5]}")
print(f"Tempo total (paralela): {end_time - start_time:.2f} segundos")
```

Q02 Determine o speedup dos dois códigos anteriores.

Q03

Um programa de computador é executado em 20 segundos por um determinado processador, sendo que 60% de seu tempo de execução é potencialmente paralelizável. Se esse programa for executado de forma paralela por 10 desses processadores, seu speedup, com duas casas decimais de precisão, será de

A) 1,56
B) 1,61
C) 2,17
D) 2,33
E) 9,20

Q04

Considerando-se que 30% das operações de um determinado programa têm de ser feitas sequencialmente, o speed up (razão entre o tempo de execução do mesmo programa por um processador sequencial e um processador paralelo) máximo que poderá ser obtido para esse programa é de, aproximadamente,

A) 0,3
B) 0,7
C) 1,43
D) 1,7
E) 3,33

Q05

Suponha que um algoritmo necessite 20 horas de processamento para completar sua execução. Considere que a parte do algoritmo não passível de paralelização demore uma hora para executar, e que 19 horas de processamento podem ser executados de forma paralela.

Assinale a alternativa que contém o potencial teórico máximo, em quantidade de vezes, de melhoria na velocidade de execução (speedup) do algoritmo em um cenário de computação paralela, independentemente da quantidade de processadores empregada, de acordo com a lei de Amdahl.

A) 4
B) 9
C) 10
D) 19
E) 20

Q06

Como se sabe, em arquiteturas computacionais com vários processadores é comum se medir o desempenho do uso de apenas um processador. Assinale a alternativa que apresenta o termo que define a relação entre o tempo consumido na execução de uma tarefa com um único processador e o tempo consumido com N processadores:

A) Escalabilidade.
B) Permuta.
C) Ingerência mista.
D) Relação de Blum.
E) Speedup.